

We filtered the data for 10 years from 4/8/2016 to 4/8/2026 - did some data cleaning in Excel, converted the 4 datasets to csv, and merged them together in SQL. Then, exported that merged dataset from SQL as csv and loaded into Jupyter Notebook to use Pandas.

Renamed columns in Excel itself - makes it easier to read (e.g. PX_LAST renamed to vix, oil_price) → renamed the credit spreads column to just credit_spreads
Renamed EMB ETF price column to just emb_price

Created a database on neon and connected to it in PgAdmin → Created tables in the database setup in PgAdmin, imported in the CSVs to create 4 different tables - VIX, uso_crude, credit_spreads, and emb_etf

Dates were written in the format mm/dd/yyyy (originally stored as text values) → converted this from text dtype into dates dtype

```
ALTER TABLE emb_etf
ALTER COLUMN "Date" TYPE date
USING to_date("Date", 'MM/DD/YYYY');
```

```
ALTER TABLE credit_spreads
ALTER COLUMN "Date" TYPE date
USING to_date("Date", 'MM/DD/YYYY');
```

```
ALTER TABLE uso_crude
ALTER COLUMN "Date" TYPE date
USING to_date("Date", 'MM/DD/YYYY');
```

```
ALTER TABLE "VIX"
ALTER COLUMN "Date" TYPE date
USING to_date("Date", 'MM/DD/YYYY');
```

Fixing the column name from Date to date

```
ALTER TABLE emb_etf RENAME COLUMN "Date" TO date;
ALTER TABLE credit_spreads RENAME COLUMN "Date" TO date;
ALTER TABLE uso_crude RENAME COLUMN "Date" TO date;
ALTER TABLE "VIX" RENAME COLUMN "Date" TO date;
```

Double checking that dtype is switched from text to date for the column containing all the dates

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'emb_etf';
```

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'credit_spreads';
```

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'uso_crude';
```

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'VIX';
```

Inner join on all 4 datasets to get only the dates that exist in all 4 of them (only the overlap)

```
CREATE TABLE merged_data AS
SELECT
    e.date,
    e.emb_price,
    c.credit_spread,
    o.oil_price,
    v.vix
FROM emb_etf e
INNER JOIN credit_spreads c
    ON e.date = c.date
INNER JOIN uso_crude o
    ON e.date = o.date
INNER JOIN "VIX" v
    ON e.date = v.date
ORDER BY e.date;
```

Inspecting results

```
SELECT *
FROM merged_data
LIMIT 20;
```

So we get 20 rows from the combined dataset - testing to see if we at least have some values that exist properly

```
SELECT COUNT(*)
FROM merged_data;
```

Result is 2513 - so we have 2,513 days worth of data over the span of 10 years

Checking for duplicates by date

```
SELECT date, COUNT(*)  
FROM merged_data  
GROUP BY date  
HAVING COUNT(*) > 1;
```

Returns nothing - so we don't have any days that have multiple credit_spreads/oil_prices etc associated with them